

ResolveIQ

Complaint Anomaly Detection, Priority Routing & Narrative RAG

A Day-by-Day Build Plan on Real CFPB Consumer Complaint Data

CFPB public API · Airflow + n8n · Postgres/pgvector · LangChain create_agent + LangGraph · Docker → Kubernetes (EKS)

How to Use This Plan

This plan mirrors the structure of the original Revenue-to-Close build, because the underlying skill stack is identical — only the domain and the data source change. If you've already built that project, large parts of this will feel like applying a known pattern to new data rather than learning from zero. That's intentional, and it's also realistic: real engineering careers are full of 'I've solved this shape of problem before, just with different inputs.'

The single biggest structural difference from the Revenue-to-Close build: there is no synthetic data phase. The CFPB Consumer Complaint Database is real, public, free, requires no API key, and updates daily. That means Phase 1 — which in the finance project was 'invent a fictional company and hand-enter its data' — becomes 'explore a real 6.6-million-plus-complaint-a-year dataset and understand its actual shape.' This saves real time, but it introduces a different kind of work: real data is messier, has actual gaps and surges, and you don't get to control its edge cases — you have to go find them.

Pacing: this plan totals roughly 16-18 weeks (4-4.5 months) at 4 hrs/day, 5 days/week — somewhat shorter than the Revenue-to-Close plan's 17 weeks, because skipping synthetic data generation removes real days of work without cutting any engineering depth. Buffer days are built into every phase; treat them as real slack, not days to work ahead, because CFPB data WILL surprise you at least once (a schema field that changes meaning over time, a sudden volume surge, an encoding quirk in older records).

Use AI assistance the same way as before: heavily for boilerplate (DAG skeletons, Pydantic schemas, Dockerfiles, K8s manifests, test scaffolding) and for explaining unfamiliar APIs. Keep ownership of the decisions that are the actual portfolio value here — your imbalance-handling strategy, your anomaly thresholds, your tool-scoping choices for the agent, and your honest read of what the narratives can and can't tell you (they're unverified consumer self-reports, not fact — see the caveat below, and keep it in mind every time you're tempted to treat a narrative as ground truth).

A Critical Framing Point: What This Data Actually Is

Read this before you start

CFPB complaint narratives are opt-in, PII-scrubbed (redactions appear as XXXX), unverified consumer self-reports — not a statistical sample of company behavior and not fact-checked by the CFPB. A company can have many complaints simply because it's large, not because it behaves worse than smaller competitors. Complaint volume also surged sharply in 2024-2025 (driven heavily by credit-reporting disputes), so any per-company or per-product trend you build must be normalized for that surge before you call anything an 'anomaly.' Bake this honesty into your agent's system prompt and into your own writeup — claiming more certainty than the data supports is exactly the kind of mistake a careful reviewer will catch, and exactly the kind of nuance that makes you look senior when you flag it yourself.

Before You Start: Accounts You'll Need

Much shorter list than the Stripe/QuickBooks project — this is the upside of building on a public dataset.

Account	Why	Notes
None for CFPB itself	The Consumer Complaint Database API is public domain, no key required	JSON/CSV/XLS responses, updates daily, full history back to 2011
AWS account	S3, RDS, EKS, ECR, CloudWatch	Set a billing alarm on day 1 — same discipline as any cloud-deployed project

Account	Why	Notes
GitHub account	Source control + GitHub Actions CI/CD	You likely have this already
Anthropic / OpenAI API key	LLM calls for the agent layer + embeddings	Pay-as-you-go; low cost until Phase 4

Project Naming

ResolveIQ is the working name from the source brief and is genuinely good — it reads as an operations/compliance product, not a toy. Stick with it, or consider:

- ResolveIQ (recommended — already well-framed)
- ComplaintLens
- TriageOps AI

Table of Contents

How to Use This Plan.....	2
A Critical Framing Point: What This Data Actually Is.....	2
Before You Start: Accounts You'll Need	2
Project Naming.....	3
Table of Contents	4
Phase 0 — Environment, Repo Foundation & First Real API Call.....	5
Phase 1 — Canonical Schema & Real Data Exploration	7
Phase 2 — Data Platform: Ingestion, Airflow, Postgres Marts	9
Week 3 — Airflow + ingestion DAGs	9
Week 4 — Canonical schema implementation + gold marts + RDS.....	10
Phase 3 — Intelligence Layer: Routing, Urgency, and Anomaly Models	12
Week 5 — Multiclass routing classifier	12
Week 6 — Timeliness / urgency classifier	13
Week 7 — Anomaly detection (volume + narrative outliers)	14
Phase 4 — Agent Layer: Narrative RAG, Citations, n8n Triage	16
Week 8 — Retrieval foundation: pgvector over real narratives	16
Week 9 — The agent: grounding, citations, defensive prompting	17
Week 10 — n8n: HITL triage with the Wait-node pattern.....	19
Week 11 — FastAPI hardening + full local stack	20
Phase 5 — Production Deployment: Kubernetes, EKS, CI/CD	21
Week 12 — Kubernetes fundamentals + manifests (local first)	21
Week 13 — EKS: the real cloud deployment	22
Week 14 — CI/CD, monitoring.....	23
Phase 6 — Portfolio Packaging.....	24
Week 15 — Demo, screenshots, metrics	24
Week 16 — Repo polish, resume, final review	25
Reference: Risk Register	26
Reference: Skills Map (same baseline, real-data context).....	27
Reference: Phase-by-Phase Timeline Summary.....	28

Phase 0 — Environment, Repo Foundation & First Real API Call

Goal: a working local dev environment, the repo skeleton with CI running, AWS account hygiene in place, and — unlike the finance project's OAuth-heavy Day 3 — an actual successful pull of real CFPB complaint data by the end of Day 2. No sandbox provisioning, no credential dance. The friction in this project shows up later (in data volume and messiness), not here.

Duration: 4 days.

Day 1 Machine setup, repo skeleton, Docker

- Confirm your toolchain: Git, Python 3.11 or 3.12 (same reasoning as before — Airflow and most current ML/LangChain libraries are tested against these), and Docker Desktop/Engine + Compose.
- Verify Docker works (docker run hello-world). Apple Silicon users: same caveat as before — watch for arm64 gaps in specific images.
- Create the GitHub repo (e.g. resolveiq-platform). Add .gitignore (Python + Node + .env).
- Lock in your repo layout now, mirroring the proven structure from the finance build but renamed for this domain:

```
resolveiq-platform/
├── infra/                # docker-compose, k8s manifests
├── ingestion/           # CFPB API client, backfill scripts
├── airflow/             # DAGs, plugins
├── api/                 # FastAPI service (agent + query layer)
├── ml/                  # training scripts, model artifacts
├── n8n/                 # exported n8n workflow JSON
├── db/                  # SQL schema, migrations (alembic)
├── tests/
├── docs/
├── .github/workflows/   # CI
├── docker-compose.yml
├── Makefile
└── README.md
```

- Set up Python virtual environments per-service (api/, ml/, ingestion/) as before, with uv or venv+pip-tools, plus a Makefile with shortcuts.

Deliverable: Repo created with the folder skeleton committed, Docker confirmed working, Makefile with at least one working target.

Day 2 Docker Compose for local infra + first real CFPB API call

- Write docker-compose.yml standing up Postgres (will hold both Airflow's metadata DB and, later, your domain schema — use separate databases inside the same instance, exactly as in the finance build) and n8n. Use named volumes.
- Bring the stack up, confirm Postgres is reachable, create an empty database (e.g. resolveiq).
- Now the genuinely fast part: hit the CFPB Consumer Complaint Database API directly. It is public, requires no key, and returns JSON. Write a 10-line Python script that does a single GET against the search/export endpoint with a small date-range filter and confirm you get real complaint records back.
- Read the API field reference (cfpb.github.io/api/ccdb/fields.html) end to end today — every field you'll model your schema around in Phase 1 is documented there, and reading it now means Phase 1's schema design isn't a guessing exercise.
- Note immediately: complaints are only published after the company responds or after 15 days, whichever is first — this means the database is not a real-time stream of submitted complaints, it's a stream of

resolved-or-aged complaints. This single fact affects how you should talk about 'real-time' anywhere later in the project, so internalize it now rather than overclaiming later.

Deliverable: *docker compose up -d brings up Postgres and n8n locally, AND a working Python script that pulls real complaint records from the live CFPB API and prints a few to the console.*

Day 3 AWS account setup + cost guardrails

- Same discipline as any cloud-deployed project: create/designate an AWS account, set a CloudWatch Billing Alarm immediately (e.g. \$20/\$50/\$100), create an IAM user (not root) for CLI use, pick one region and stick to it.
- Create one S3 bucket now for the raw/bronze landing zone, e.g. resolveiq-raw-<yourname>.
- Do not provision RDS or EKS yet — same reasoning as before, account hygiene first, real infra only once a local pipeline proves itself.

Deliverable: *AWS account with billing alarm, IAM user configured locally via CLI, one S3 bucket created.*

Day 4 CI skeleton, ADR, buffer

- Add a GitHub Actions workflow running lint + pytest on every push, even against a trivial test.
- Write ADR-0001 in docs/adr/: capture why CFPB over the other two candidate datasets (Olist, MIMIC) for this build — reusable answer for interviews ('public domain license, no credentialing wait, genuine narrative+structured pairing, real production triage framing'). Also capture your initial stack choices (Postgres+pgvector, Airflow+n8n split, LangChain create_agent) with the same one-line rationale style as the finance project's ADR.
- Use slack time to read 1-2 real CFPB complaint narratives end to end (browse the public search UI at consumerfinance.gov/data-research/consumer-complaints/search) to get a feel for the actual text you'll be working with — tone, length, redaction patterns (XXXX). This is the equivalent of the finance project's 'manually trace a customer through both systems' exercise: cheap, and it pays off every time you write a prompt or a classifier feature later.

Deliverable: *CI pipeline green, ADR-0001 committed, and you've personally read enough real complaint narratives to have an intuition for what the text actually looks like.*

Phase 1 — Canonical Schema & Real Data Exploration

Goal: a designed canonical schema mapping the CFPB's real fields, and a genuine, hands-on understanding of what's actually in this dataset — its categories, its messiness, its known surges, its missingness patterns — before you write a single transformation. This replaces the finance project's 'build a fictional company' phase: instead of inventing edge cases, you go find the real ones.

Duration: 5 days.

Day 5 Bulk download + first real exploration pass

- Pull a substantial historical slice via the bulk CSV/JSON download (not just the API) — this is faster for initial exploration than paginating the API and lets you work offline in a notebook. Start with 2-3 recent full years rather than the entire 2011-present history; you can always backfill further once your pipeline is proven.
- Load it into a pandas dataframe and actually look: `value_counts()` on Product, Sub-product, Issue, Company, State, Submitted via, Company response, Timely response. Get a feel for the cardinality (how many distinct companies? products? — this matters for your schema's foreign-key/dimension design).
- Specifically check: what fraction of complaints have a non-null consumer narrative? (This is your RAG corpus size — know it now, not in Phase 4.) What does the distribution of narrative length look like?
- Plot complaint volume over time at a daily/monthly grain. You will see the 2024-2025 surge mentioned in the source brief — confirm it yourself rather than taking it on faith, and look at which Product/Issue categories are driving it (the brief notes credit-reporting disputes as the major driver — verify this in your own slice).

Deliverable: A notebook (kept in `docs/exploration/` or similar, not deleted afterward) showing real value counts, narrative-presence rate, and a volume-over-time plot that visibly shows the known surge — annotated with your own observations.

Day 6 Design the canonical schema

- Design three gold-layer tables matching the source brief's recommended schema, but make the design decisions yours rather than copy-pasting: `mart_complaint` (`complaint_id` PK, `date_received`, `product`, `subproduct`, `issue`, `subissue`, `company`, `state`, `zip3`, `submitted_via`, `date_sent_to_company`, `company_response`, `timely_response`, `response_lag_days`), `mart_complaint_narrative` (`complaint_id` PK, `narrative_text`, `narrative_len`, `has_narrative`), `mart_daily_volume` (`date`, `product`, `issue`, `complaint_count`).
- Decide explicitly: `zip3` not full ZIP (the source notes this is the level of geographic granularity CFPB actually exposes — full ZIP isn't published, so don't design a schema that assumes it).
- As with the finance project, keep a `raw_payload` JSONB column on the staging/bronze-adjacent tables so every normalized row traces back to its original API response — same audit-trail discipline, same reasoning (you must always be able to answer 'where did this number come from').
- Decide your `response_lag_days` computation precisely (`date_sent_to_company` minus `date_received`, in days) and write down edge cases: what happens when `date_sent_to_company` is null (in-progress complaints)? Handle this explicitly rather than letting it silently become a NaN that breaks a later model.

Deliverable: `docs/schema-notes.md` plus an ERD sketch (`docs/erd-v1.png`) for the three gold tables, with explicit decisions written down for the genuinely ambiguous cases (null response dates, `zip3` granularity).

Day 7 Categorical normalization research

- CFPB's Product/Sub-product/Issue/Sub-issue taxonomy has changed over time (categories get renamed, split, merged across the database's history back to 2011) — this is the real-data equivalent of the finance project's 'Stripe vs QuickBooks status enum' mapping problem. Pull the full distinct list of Product values

across your historical slice and check for near-duplicates or renames (e.g. category names that clearly refer to the same thing but changed wording in different years).

- Decide your normalization strategy now: do you map historical category names to current ones (more useful for trend analysis, more work), or keep them as-is and let your time-series mart show known regime breaks (less work, slightly messier story)? Write the decision down — this is exactly the kind of judgment call worth being able to explain in an interview.
- Check the Company field similarly: company names in CFPB data are not normalized (the same company may appear under slightly different name strings) — decide now whether you'll do basic fuzzy-grouping for your per-company analysis or accept the noise; for a portfolio project, accepting and documenting the noise is a legitimate choice as long as you say so.

Deliverable: *A written categorical-normalization decision (docs/categorical-mapping.md) covering Product/Issue taxonomy drift and company-name noise, with your chosen approach and rationale.*

Day 8 Read narratives closely — build your editorial sense

- Sample 30-50 real narratives across different Products (mortgage, credit reporting, debt collection, credit card, etc.) and read them properly — not skim. You're building the same kind of domain intuition the finance project's manual reconciliation trace gave you, except here it's 'what does an urgent complaint actually sound like versus a routine one,' which you'll need for labeling/feature intuition in Phase 3 and for writing a believable agent system prompt in Phase 4.
- Note recurring structural patterns: which narratives mention specific dollar amounts, dates, or repeated contact attempts (these often correlate with the more serious cases); how redaction (XXXX) appears and how much it disrupts readability; whether certain Products' complaints read distinctly differently from others.
- Write a short docs/narrative-notes.md capturing this — it will directly inform feature engineering in Phase 3 and prompt design in Phase 4.

Deliverable: *docs/narrative-notes.md with genuine qualitative observations from reading real complaint text, not generic assumptions.*

Day 9 Buffer / finalize schema based on what you learned

- Revise the Day 6 schema based on anything Days 7-8 taught you (there will be at least one surprise — a field that's less clean than expected, a category split you didn't anticipate).
- Catch up on any slippage from Days 5-8 — this phase has a real risk of running long simply because reading real data closely takes longer than people expect; that's fine, it's the actual value of this phase.

Deliverable: *A finalized, data-informed schema (ERD v2) ready to implement as real DDL in Phase 2.*

Phase 2 — Data Platform: Ingestion, Airflow, Postgres Marts

Goal: an automated, idempotent pipeline pulling real CFPB data daily (incremental) plus a working historical backfill DAG reaching back as far as you choose, landing raw JSON in S3, normalizing into your canonical schema in RDS Postgres, and building the three gold marts. This is the direct analog of the finance project's Phase 2, with one real difference: a single source (CFPB) instead of two (Stripe+QuickBooks), which simplifies the identity-mapping problem that ate a full day in the original plan — there's no cross-system customer-matching here.

Duration: 2 weeks (10 days) — shorter than the finance project's 3 weeks for exactly this reason.

Week 3 — Airflow + ingestion DAGs

Day 10 Get Airflow running locally

- Same approach as the finance project: use Airflow's official docker-compose.yaml as your base, merge into your existing compose file, point its metadata DB at a separate database inside your shared Postgres instance, set LocalExecutor for local dev.
- Confirm the webserver UI loads at localhost:8080 and disable example DAGs.

Deliverable: Airflow running locally via Docker Compose, reachable in the browser, metadata DB isolated from your domain schema.

Day 11 Bronze ingestion DAG — incremental daily pull

- Write ingestion/cfpb_client.py wrapping the CFPB search/export API with pagination handled (the API returns paged JSON; you'll need to loop through pages for any date range with meaningful volume).
- Write the bronze_ingest Airflow DAG: daily-scheduled, pulls complaints with Date received in the current logical date's window, lands raw JSON to S3 at s3://resolveiq-raw/cfpb/dt=YYYY-MM-DD/.
- Use the TaskFlow API (@dag/@task) as in the finance project. Make it idempotent: re-running for a given logical date overwrites that date's partition cleanly.
- Add basic retry/backoff — the CFPB API is public and unauthenticated, which is convenient, but also means no documented per-key rate limit to rely on; be a good citizen with reasonable request pacing regardless.

Deliverable: A working bronze_ingest DAG pulling real daily CFPB data into S3, idempotent and re-runnable.

Day 12 Backfill DAG — historical data

- Write a separate backfill_ingest DAG (or parameterize the same DAG for backfill mode) that paginates historical data in date-range chunks back to 2011, landing each chunk the same way as the daily DAG.
- Decide your actual backfill depth deliberately rather than defaulting to 'all the way to 2011' — going back 2-3 years is plenty for meaningful trend/anomaly analysis and dramatically reduces both ingestion time and the volume you'll be processing in every later phase. Write this decision down (it's a legitimate scoping call, and saying 'I deliberately bounded historical depth to keep iteration fast' is a better interview answer than silently doing it without acknowledging the tradeoff).
- Run the backfill for your chosen window and confirm row counts in S3 are sane (spot-check against the known public total — millions of rows for multi-year windows, so confirm your pagination isn't silently dropping pages).

Deliverable: A backfill DAG that has successfully populated S3 with your chosen historical window, with row counts validated against expectations.

Day 13 Bronze-to-silver: staging tables

- Design and create staging tables mirroring the CFPB's raw field names (mostly JSONB + a few extracted keys like `complaint_id` and `date_received`) — same bronze/silver pattern as the finance project.
- Write the `silver_transform` DAG step that bulk-loads from S3 into staging via COPY/batched inserts (not row-by-row), with task dependencies ensuring it only runs after a successful bronze pull.

Deliverable: *Staging tables populated from S3 via an Airflow-orchestrated bulk load, with correct task dependencies.*

Day 14 Data-quality checks + buffer

- Add explicit Airflow data-quality tasks per the source brief: assert non-null Complaint ID, valid Product enum membership (against your Day 7 normalized taxonomy), and monotonic dates (`date_sent_to_company` $\geq$ `date_received` where both are present) — fail loudly, don't silently load bad rows.
- Run a clean backfill re-run to confirm idempotency holds under Airflow's actual rerun/backfill mechanics, not just a single fresh run.
- Catch up on slippage from Days 10-13.

Deliverable: *Both DAGs pass a clean backfill re-run with data-quality checks in place and failing loudly on bad input.*

Week 4 — Canonical schema implementation + gold marts + RDS

Day 15 Implement canonical schema + transformation DAG

- Translate your Phase 1 ERD v2 into real Alembic migrations: `mart_complaint`, `mart_complaint_narrative`, `mart_daily_volume`, plus a `complaint_exceptions` table you'll need in Phase 3 (mirroring the finance project's `reconciliation_exceptions` table — here it'll hold anomaly/urgency flags instead).
- Write the `gold_marts` transformation DAG step mapping staging rows into the canonical tables, applying your Day 7 categorical normalization decisions and your Day 6 null-handling rules for `response_lag_days`.
- Add appropriate indexes now for known query patterns: date ranges, Product/Issue combinations, Company lookups.

Deliverable: *Canonical schema implemented via Alembic migrations, with a working transformation DAG populating it from staging.*

Day 16 Build `mart_daily_volume` + first real exploration of trend shape

- Populate `mart_daily_volume` (`date`, `product`, `issue`, `complaint_count`) — this is your anomaly-detection substrate for Phase 3, so get it right: aggregate at a grain that's actually useful (daily is probably right, but check it isn't too sparse for less common Product/Issue combinations — you may need a coarser fallback grain for low-volume categories).
- Query it and look at real shapes: does Product X show weekly seasonality? Are there real, dateable events visible in the surge data (e.g., does a known regulatory change or major news event in your window show up as a visible spike)? This kind of correlation-with-real-events is exactly what gives your eventual anomaly detection credibility — knowing what a 'real, explicable' spike looks like helps you sanity-check what your model flags later.

Deliverable: *`mart_daily_volume` populated and queried, with at least one real, identifiable pattern (seasonality or a dateable surge) found and noted in docs/.*

Day 17 Move to RDS

- Provision an RDS PostgreSQL instance (smallest viable class), enable pgvector now even though Phase 4 is when you'll use it, re-point Airflow connections at RDS, re-run migrations.
- Lock down security groups to your IP only.

Deliverable: *RDS instance live, schema migrated, pgvector enabled, Airflow writing to it successfully.*

Day 18 Re-run full pipeline against RDS + tests

- Do a full clean backfill against RDS and fix whatever breaks (connection strings, timeouts, IAM/network).
- Write pytest tests for your transformation logic using real fixture data pulled from a known date range — assert specific rows transform correctly, including your null-handling and categorical-normalization edge cases from Phase 1.
- Add these tests to CI.

Deliverable: *A clean full-pipeline backfill succeeding against RDS, with transformation unit tests running in CI.*

Day 19 Buffer / Phase 2 close-out + documentation

- Write docs/data-platform.md documenting the bronze/silver/gold flow, the DAGs, the backfill depth decision, and how to re-run things — same discipline as the finance project, and this becomes core material for your eventual writeup.
- Use remaining slack to catch up on anything from Week 3-4, and optionally add Airflow failure alerting now rather than waiting.

Deliverable: *Phase 2 fully closed out: ingestion, backfill, transformation, and RDS migration all working and tested, documented, with no open debt.*

Phase 3 — Intelligence Layer: Routing, Urgency, and Anomaly Models

Goal: three trained, evaluated models — a multiclass Product/Issue text classifier for auto-routing, a timeliness/urgency classifier predicting which complaints risk an untimely response, and an anomaly detector flagging unusual volume spikes and outlier narratives — all wired into Postgres via Airflow so the agent layer has real signal to reason over. Same structure as the finance project's Phase 3, with real, naturally-occurring class imbalance instead of imbalance you'd have to deliberately seed.

Duration: 3 weeks (15 days).

The imbalance reality, stated plainly

Real CFPB data hands you genuine rare-event problems with no synthetic shortcuts: 'not timely' responses and true anomalies are both naturally rare. The standard toolkit applies — class weighting, gradient boosting with scale_pos_weight-style handling, isolation forests for unsupervised detection, SMOTE/undersampling where appropriate, and threshold tuning evaluated on PR-AUC and precision/recall rather than plain accuracy, which would be misleadingly high on an imbalanced target by default.

Week 5 — Multiclass routing classifier

Day 20 Define the routing task + build the training set

- Task: given a complaint narrative, predict its Product (and optionally Issue) — the real use case being 'auto-route mis-categorized complaints,' since in practice some submitted complaints are filed under the wrong category and a model that reads the narrative can catch this.
- Build your train/test set from mart_complaint joined to mart_complaint_narrative, filtered to has_narrative=true. Check class balance across Product categories — some (credit reporting, debt collection) will dominate by sheer volume; decide whether to downsample the majority classes for training balance or just weight the loss function, and write down which you chose and why.
- Hold out a genuine temporal slice for testing (e.g., train on everything before a cutoff date, test on complaints after it) rather than a random split — this avoids leakage from near-duplicate complaints about the same ongoing issue landing in both train and test, and is a more honest estimate of how the model performs on genuinely new complaints.

Deliverable: A labeled training set with a temporal train/test split and a documented class-balance decision.

Day 21 Baseline model: TF-IDF + linear classifier

- Train a TF-IDF + logistic regression (or linear SVM) baseline first — fast to train, genuinely competitive on this kind of categorical text-routing task, and fully explainable (you can inspect which terms drive a given prediction, which matters when you later need to explain misroutes).
- Evaluate with macro-F1 (per the source brief) since you care about reasonable performance across all Product categories, not just the dominant ones — macro-F1 penalizes a model that nails the big categories and ignores small ones, which accuracy alone would hide.

Deliverable: A baseline TF-IDF model with a macro-F1 score, plus a few inspected examples of which terms drive correct/incorrect predictions.

Day 22 Try a stronger model + compare

- Try a fine-tuned transformer (a small one — DistilBERT-scale is plenty for this task and won't demand GPU infrastructure you don't have) or a stronger gradient-boosted approach on richer features, and compare against the Day 21 baseline honestly.

- Write the comparison down with a real rationale for your final choice — the source brief explicitly allows either path ('TF-IDF + linear model or a fine-tuned transformer'), so there's no single right answer; what matters is that you can defend yours.

Deliverable: *A documented model comparison (docs/model-comparison-routing.md) and a finalized routing model.*

Day 23 Error analysis

- Pull the actual misclassified examples and read them — this is the highest-value hour of the week. Real misroutes often reveal genuinely ambiguous cases (a complaint that legitimately touches two Product categories) rather than model failure, and being able to say that in an interview is a stronger signal than a high accuracy number alone.
- Document 5-10 concrete misclassification examples with your read on why each happened, in docs/error-analysis-routing.md.

Deliverable: *docs/error-analysis-routing.md with real, specific misclassification examples and your analysis.*

Day 24 Wire into pipeline + buffer

- Add predicted_product, predicted_issue, prediction_confidence columns and wire model inference into Postgres via an Airflow task, scored on new incoming complaints from the daily ingestion DAG.
- Catch up on any slippage from Days 20-23.

Deliverable: *Routing predictions flowing into Postgres automatically as part of the daily pipeline.*

Week 6 — Timeliness / urgency classifier

Day 25 Define the target + features

- Target: predict whether a complaint's company response will be 'not timely' (using the Timely response? field as ground truth for historical, resolved complaints) — useful for flagging at-risk complaints proactively rather than discovering lateness after the fact.
- Engineer features: Product/Issue category (some categories may structurally run slower), Company (some companies are slower responders — but normalize for company size/volume per the source brief's caveat about company-level comparisons), State, Submitted via, day-of-week/month received (operational load effects), narrative length and presence.
- Check the actual positive rate for 'not timely' in your data — this number itself is a finding worth recording, and it tells you how aggressively you need to handle imbalance.

Deliverable: *A feature table for the urgency/timeliness target with the real positive rate documented.*

Day 26 Train + handle imbalance

- Train a gradient-boosting classifier (XGBoost/LightGBM) with class weighting as the primary approach, per the source brief.
- If the positive class is severely rare, also try threshold tuning on the PR curve explicitly rather than using the default 0.5 cutoff — pick an operating point and justify it in business terms (e.g., 'we'd rather over-flag for human review than miss a likely-late case, given review capacity constraints').

Deliverable: *A trained urgency classifier with an explicitly chosen, justified decision threshold.*

Day 27 Evaluate properly

- Evaluate with precision/recall/PR-AUC on the rare positive class specifically (per the source brief) — not accuracy, which will look artificially high given the class imbalance.

- Write `ml/urgency/evaluate.py` as a standalone, re-runnable evaluation script, same pattern as the finance project's evaluation harnesses.

Deliverable: *ml/urgency/evaluate.py* producing a concrete PR-AUC/precision/recall report, re-runnable on demand.

Day 28 SHAP explainability

- Add SHAP value computation for the urgency model — per-prediction explainability matters here for the same reason it mattered in the finance project's forecasting model: a human reviewer (or your future agent) needs to be able to say **why** a complaint was flagged, not just that it was.
- Confirm SHAP outputs make intuitive sense on a handful of real flagged complaints (read 3-5 of them and check the top SHAP features against your own judgment of why they look urgent).

Deliverable: *SHAP explanations wired into the urgency model's output, sanity-checked against real examples.*

Day 29 Wire into pipeline + buffer

- Add `urgency_score`, `is_flagged_urgent`, `shap_top_factors` columns and wire scoring into the Airflow pipeline.
- Catch up on slippage.

Deliverable: *Urgency scores and explanations flowing into Postgres automatically.*

Week 7 — Anomaly detection (volume + narrative outliers)

Day 30 Time-series anomaly detection on `mart_daily_volume`

- Build a time-series anomaly detector over `mart_daily_volume`, per company/product where volume supports it — a statistical approach (rolling mean/std with z-score thresholds, or STL decomposition residuals) is a legitimate, explainable starting point; don't reach for something exotic before trying the simple thing.
- Critical per the source brief's caveat: normalize for the known 2024-2025 volume surge before flagging anything as anomalous — a naive model will flag the entire surge period as 'anomalous' when it's actually a real, sustained shift, not a point anomaly. Decide explicitly how you'll handle this (e.g., detrend first, or model the surge as a regime change rather than noise) and document the decision.

Deliverable: *A time-series anomaly model that does NOT simply flag the entire known surge period, with the surge-handling decision documented.*

Day 31 Isolation forest on narrative embeddings

- Generate embeddings for a sample of narratives (you'll build the full embedding pipeline properly in Phase 4 for RAG — here, reuse a lightweight version) and run an isolation forest to find outlier narratives — complaints whose text is unusually different from the surrounding cluster for their Product/Issue category.
- Per the source brief, tune contamination by sweeping a small range (e.g., 0.01-0.05) and picking the operating point via a precision/recall tradeoff you assess by actually reading the flagged outliers.

Deliverable: *An isolation-forest narrative-outlier detector with a deliberately tuned contamination parameter, validated by reading actual flagged examples.*

Day 32 Combine into `complaint_exceptions` + `precision@k`

- Populate the `complaint_exceptions` table (created in Phase 2) with both volume-anomaly flags (per company/product/period) and narrative-outlier flags (per complaint), each with a score and a reason.

- Evaluate with precision@k rather than raw recall, per the source brief's reasoning: investigators have limited capacity, so ranking quality for the top-k flagged items matters more than catching every possible anomaly. Compute precision@10, @25, @50 and report them.

Deliverable: *complaint_exceptions populated with both anomaly types, evaluated with precision@k, the metric matched to the real operational constraint.*

Day 33 Wire into pipeline + write the model evaluation summary

- Wire anomaly scoring into the Airflow pipeline as a scheduled task.
- Write docs/model-evaluation.md consolidating all three models' metrics, what they mean in operational terms, and known limitations — same discipline as the finance project's equivalent doc, and this is a non-negotiable per the original brief's own evaluation-first philosophy.

Deliverable: *docs/model-evaluation.md complete, anomaly detection running automatically in the pipeline.*

Day 34 Buffer / Phase 3 close-out

- Catch up on any slippage from Weeks 5-7. Re-read your three evaluation docs and confirm each model's output actually answers a real operational question (not just 'the model runs') — this is the same discipline the finance project applied to its non-negotiables.

Deliverable: *Phase 3 fully closed out: all three models trained, evaluated, documented, and running automatically with their outputs queryable in Postgres.*

Phase 4 — Agent Layer: Narrative RAG, Citations, n8n Triage

Goal: an agent that answers real analyst questions — 'What themes drive mortgage servicing complaints against Company Y this month?', 'Show me anomalous complaint clusters this week and cite examples,' 'Summarize the most common debt-collection sub-issues in Texas' — grounded in real complaint narratives with enforced, verbatim citation back to specific complaint IDs. Every flagged anomaly or urgent complaint routes through an n8n human-in-the-loop approval gate before anything is treated as actioned. This is the heaviest phase, same as in the finance project.

Duration: 4 weeks (20 days).

Current LangChain API notes — read before Day 35

LangChain v1.0 (released October 22, 2025) made `create_agent` (from the `langchain` package, running on the LangGraph runtime) the current, correct API — it supersedes both the legacy `AgentExecutor` and the `langgraph.prebuilt.create_react_agent` shortcut. For structured output, use `response_format` with an explicit `ToolStrategy` or `ProviderStrategy` wrapper rather than a bare schema — LangChain's own docs have had documented ambiguity here, and being explicit avoids it. One real, current friction point worth knowing up front: there are open reports of `ToolStrategy` and tool-calling interacting awkwardly when combined with extended thinking on Anthropic models specifically (forcing `tool_choice` can conflict with thinking being enabled) — if you hit this, the workaround is to either disable extended thinking for the structured-output call or use `ProviderStrategy` where supported. Don't burn a day debugging this blind; it's a known issue.

Week 8 — Retrieval foundation: pgvector over real narratives

Day 35 Chunking strategy for real complaint narratives

- Unlike the finance project's short synthetic contracts, you now have real narrative text of varying length — some are a sentence, some are multiple paragraphs. Use `RecursiveCharacterTextSplitter` with `chunk_size` around 400-1000 characters depending on what your Day 8 (Phase 1) reading taught you about typical narrative length; smaller chunks (~400 chars) tend to work well for shorter consumer-complaint text per the source brief.
- Set `add_start_index=True` when chunking — this retains a character offset you can use as part of your citation anchor, and is a detail the source brief specifically calls out as worth getting right from the start.
- Decide your chunking scope: chunk only narratives with `has_narrative=true` (most complaints won't have one), and confirm your corpus size (number of chunks) is something you actually computed back in Phase 1, not a guess.

Deliverable: A chunking pipeline applied to real narrative text, with start-index offsets retained, and a confirmed total chunk count.

Day 36 Embedding + pgvector storage with metadata filtering

- Generate embeddings (1536-dim via a standard embedding model) for every chunk and store in a chunks table: (chunk_id, complaint_id, product, issue, company, state, chunk_index, content, embedding vector(1536), metadata jsonb) — the metadata columns matter because you'll need filtered retrieval (e.g., 'only search within mortgage servicing complaints against Company Y'), not just open similarity search.
- Create an HNSW cosine index (`USING hnsw (embedding vector_cosine_ops) WITH (m=16, ef_construction=64)`) — these are reasonable, commonly-used defaults; note for your own ADR that the specific parameter values come from engineering convention rather than official LangChain documentation, so you're making a defensible but not gospel choice.

- If your corpus exceeds roughly 1M chunks and query latency starts exceeding ~50ms at p95, that's your signal to consider pgvectorscale/DiskANN or a dedicated vector store — note this threshold now so you know what to watch for, even if you don't hit it at this project's scale.

Deliverable: All real narrative chunks embedded and stored in pgvector with an HNSW index, metadata columns populated for filtered retrieval.

Day 37 Build and test the retrieval function with metadata filters

- Write `retrieve_narratives(filter, query, k)` as a standalone, framework-independent function performing filtered pgvector similarity search (e.g., `similarity_search(query, k=4, filter={"product": "Mortgage"})`).
- Test it with real queries against real data: search for 'mortgage servicing complaints about late fees' filtered to `Product=Mortgage` and check the returned chunks are genuinely on-topic — this is your equivalent of the finance project's manual retrieval sanity check, except now you're validating against real, sometimes-messy consumer text rather than clean synthetic documents.

Deliverable: A tested, framework-independent retrieval function with metadata filtering, validated against real queries.

Day 38 Build the SQL tool functions

- Write the scoped tool functions per the source brief: `get_complaint(complaint_id)`, `get_volume_trend(product, issue, window)`, `get_anomaly_flags(window)`, plus `get_company_complaint_summary(company, product, window)` if useful for the example questions.
- Same discipline as the finance project: no raw SQL exposed to the agent, only parameterized, typed tool functions returning structured data.

Deliverable: 4-5 scoped, tested SQL tool functions covering the example analyst questions from the brief.

Day 39 Buffer / corpus and tool polish

- Fix any schema/tool gaps Day 38 revealed. Catch up on slippage from Days 35-38.

Deliverable: All retrieval and SQL tools working and tested independently of any agent framework.

Week 9 — The agent: grounding, citations, defensive prompting

Day 40 Wire the agent with `create_agent + content_and_artifact` retrieval

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool(response_format="content_and_artifact")
def retrieve_narratives(complaint_filter: dict, query: str):
    docs = vector_store.similarity_search(query, k=4, filter=complaint_filter)
    content = "\n\n".join(
        f"Source: complaint_id={d.metadata['complaint_id']}\n{d.page_content}"
        for d in docs
    )
    return content, docs  # (model-facing string, raw artifacts)

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[get_complaint, get_volume_trend, get_anomaly_flags, retrieve_narratives],
    system_prompt=ANALYST_SYSTEM_PROMPT,
)
```

- Use `@tool(response_format="content_and_artifact")` on your retrieval tool, per the source brief and LangChain's official v1 RAG guide — this attaches the raw retrieved Document objects as artifacts on the

ToolMessage, separate from the stringified content sent to the model, which is exactly what you need for later citation verification.

- Write a defensive system prompt that explicitly instructs the model to treat retrieved context as data only (ignore any instructions contained within retrieved narrative text — consumer narratives are uncontrolled user-generated text, so this is a genuine prompt-injection-adjacent concern, not paranoia) and to say it doesn't know if evidence is absent rather than fabricating an answer.
- Test manually against the three example questions from the brief; iterate until answers are grounded and correctly call the right tools.

Deliverable: A working `create_agent` instance correctly answering the example questions, using `content_and_artifact` retrieval, with a defensive system prompt guarding against injected instructions in narrative text.

Day 41 Enforced citation schema

- Define a `QuotedAnswer` Pydantic schema: `answer: str` plus `citations: list[Citation]`, where `Citation` has `source_id: int` and `quote: str` (described explicitly as 'the VERBATIM quote from the specified source that justifies the answer').
- Wire it via `response_format=ToolStrategy(QuotedAnswer)` (or `ProviderStrategy` if using a model/provider with native structured-output support) — be explicit about which strategy, per the friction note at the top of this phase.
- Test that every answer the agent produces now comes back with real `complaint_id` references and verbatim quotes you can manually verify against the source narrative — actually check a handful by hand.

Deliverable: Every agent answer returns a structured `QuotedAnswer` with verbatim citations, manually spot-checked for accuracy against source narratives.

Day 42 Persistence + audit trail

- Add a Postgres-backed `LangGraph` checkpoint for conversation persistence across restarts, same as the finance project.
- Implement the audit log (`agent_audit_log`: tool calls, arguments, results, citations returned, timestamp) — same non-negotiable principle as before: every agent action must be traceable.

Deliverable: Agent conversations persist across restarts; every tool call and citation is logged to `agent_audit_log`.

Day 43 Handle the un-verified-narrative caveat explicitly in the agent's behavior

- This is specific to this project and worth a dedicated day: narratives are unverified consumer self-reports, not fact. Extend the system prompt and a wrapping response template so the agent's answers about 'what consumers report' are phrased as such ('consumers reported X', not 'Company Y did X') — this is a real, substantive design choice, not boilerplate disclaimer text, and it's exactly the kind of judgment that separates a thoughtful build from a careless one.
- Test this explicitly: ask the agent a leading question implying a company did something wrong, and confirm its answer correctly hedges to 'consumers reported...' framing rather than asserting it as established fact.

Deliverable: A demonstrated test case where the agent correctly refuses to assert unverified narrative content as fact, with the framing in its system prompt to back it up.

Day 44 Buffer / agent evaluation pass

- Build a 10-15 question eval set (the three brief examples plus variations) with a rubric: grounded in real retrieved data, cites correctly, appropriately hedges on unverified claims, says 'I don't know' when evidence is absent. Score the agent against it manually.
- Catch up on slippage.

Deliverable: *docs/agent-eval.md with a written rubric and scored results, including at least one explicit check for appropriate hedging on unverified narrative claims.*

Week 10 — n8n: HITL triage with the Wait-node pattern

Day 45 Design the triage workflow

- Per the source brief: anomaly flags and predicted-urgent complaints should trigger an n8n workflow routing the case to a human investigator with Approve-escalation / Reassign / Dismiss actions, a context-rich message, and audit logging on every decision.
- Design this using n8n's Wait node + resume-webhook pattern (\$execution.resumeUrl) — note the source brief flags that n8n v2.0 specifically fixed a sub-workflow Wait-node bug, so if you're on an older n8n version and HITL gates inside agent-run sub-workflows behave unreliably, that's a known, documented issue, not something wrong with your workflow logic; update n8n first before debugging your own design.
- Sketch timeout behavior explicitly: high-severity flags auto-escalate on timeout, low-severity ones auto-shelve — write this logic down before building it.

Deliverable: *A written n8n workflow design covering the Wait-node approval pattern and explicit timeout/escalation behavior.*

Day 46 Build the n8n triage workflow

- Build it in the n8n UI: webhook trigger (called when your pipeline flags an urgent/anomalous complaint) -> a node constructing a context-rich message (the brief mentions building this with fromAI() variables, letting the agent populate parts of the notification dynamically) -> Wait node -> resume on Approve/Reassign/Dismiss -> callback into your API.
- Export and commit the workflow JSON to n8n/workflows/.

Deliverable: *A working n8n triage workflow, exported as versioned JSON, that notifies, waits, and resumes correctly.*

Day 47 Wire the FastAPI service to n8n and back

- Stand up/extend FastAPI: an endpoint that, when a complaint crosses the urgency/anomaly threshold, calls the n8n webhook to start triage.
- Add the reverse endpoint n8n calls on Approve/Reassign/Dismiss, updating Postgres and the audit log accordingly.

Deliverable: *A full round-trip: pipeline flags a complaint -> n8n triage fires -> decision made -> API updates state -> audit log reflects the full chain.*

Day 48 Timeout/escalation behavior

- Implement the auto-escalate-high-severity / auto-shelve-low-severity timeout behavior designed on Day 45 — test it by deliberately not responding to a triggered triage and confirming the correct timeout action fires.

Deliverable: *Demonstrated timeout behavior: a deliberately ignored triage case correctly escalates (if high-severity) or auto-shelves (if low-severity).*

Day 49 Buffer / full HITL scenario test

- Run a complete realistic scenario: a real anomalous volume spike or a real flagged urgent complaint from your actual data triggers triage, gets approved/dismissed, and the full audit trail is correct end to end.
- Catch up on slippage.

Deliverable: *A full human-in-the-loop scenario, using real flagged data, working end-to-end and demoable.*

Week 11 — FastAPI hardening + full local stack

Day 50 Harden the FastAPI service

- Typed Pydantic request/response models for every endpoint, basic API-key auth, structured error handling, request logging, auto-generated OpenAPI docs.

Deliverable: *A hardened FastAPI service with typed models, basic auth, and OpenAPI docs.*

Day 51 Streaming responses

- Add streaming for the agent question-answering endpoint so responses appear progressively rather than blocking.

Deliverable: *The agent endpoint streams partial output.*

Day 52 Containerize + full local stack

- Write a production Dockerfile for the API service, add to docker-compose.yml, confirm the full local stack (Postgres, Airflow, n8n, API) comes up together with one command and that data flows end to end.

Deliverable: *docker compose up -d brings up the entire local platform together, verified working end to end.*

Day 53 Integration tests across the stack

- Write integration tests (pytest + test client) exercising the API end-to-end against a test database: ask a question, confirm a grounded cited answer, confirm an audit log entry.
- Add to CI with a Postgres service container that has pgvector available (GitHub Actions supports service containers for this).

Deliverable: *Integration tests running in CI against a real test Postgres instance with pgvector.*

Day 54 Buffer / Phase 4 close-out

- Treat this buffer day as real — this is the most complex phase. If Day 44's agent eval was weak, fix prompts/tools now rather than carrying a wobbly agent forward.
- Write docs/agent-layer.md summarizing tools, retrieval, citation enforcement, audit trail, and the HITL flow — core material for your eventual portfolio writeup.

Deliverable: *Phase 4 fully closed out: agent answers all example questions with enforced citations, every action is audited, triage routes through n8n, and the full local stack runs as one unit.*

Phase 5 — Production Deployment: Kubernetes, EKS, CI/CD

Goal: every service running on EKS, deployed via CI/CD, with isolated node groups separating batch anomaly-detection jobs from low-latency agent inference — a distinction the source brief calls out specifically for this project, and a genuinely good Kubernetes story to be able to explain (most candidates can say 'I deployed to Kubernetes'; fewer can say 'I deliberately isolated batch and latency-sensitive workloads onto separate node pools and here's why').

Duration: 3 weeks (15 days) — same structure as the finance project's Phase 5.

Cost discipline reminder

Same as always: EKS bills hourly for the control plane plus worker node costs, and forgetting to tear a cluster down is the most common way a portfolio project quietly costs real money. Revisit your Phase 0 billing alarm now and consider lowering the threshold temporarily during this phase.

Week 12 — Kubernetes fundamentals + manifests (local first)

Day 55 Local Kubernetes cluster

- Install kind or minikube locally, confirm kubectl connects. Same reasoning as before: debug manifests locally before paying for cloud cluster time.

***Deliverable:** A local kind/minikube cluster running, kubectl get nodes healthy.*

Day 56 API service manifests

- Write Deployment + Service manifests for the FastAPI service: image, resource requests/limits, env vars from Secrets, readiness/liveness probes on a /health endpoint.
- Apply locally, confirm healthy via port-forward.

***Deliverable:** FastAPI service running as a Kubernetes Deployment locally, passing health checks.*

Day 57 n8n manifests + Postgres decision

- Write Deployment + Service + PVC for n8n.
- Plan to use RDS in production for Postgres (not in-cluster) — same ADR reasoning as the finance project; write it down once more here so the decision is documented per-project, not just inherited silently.

***Deliverable:** n8n running on the local cluster with persistent storage.*

Day 58 Airflow on Kubernetes via Helm

- Install the official Airflow Helm chart locally, pointed at RDS (or local Postgres for the dry run), confirm webserver/scheduler/triggerer come up.
- Choose KubernetesExecutor for the same reasons as the finance project: clean resource isolation per task, a more interesting and explainable story than CeleryExecutor for a portfolio-scale project.

***Deliverable:** Airflow running on the local cluster via Helm with KubernetesExecutor, at least one DAG run completing with pod-per-task.*

Day 59 Design node-group isolation (planning, not yet applying)

- This is the project-specific addition the source brief calls for: plan two logical node groups — one for batch anomaly-detection/training jobs (can tolerate higher latency, benefits from larger/cheaper instances scheduled less frequently) and one for low-latency agent inference (needs to stay warm and responsive for interactive queries).

- Sketch how you'll express this in Kubernetes terms: node taints/labels on each group, and matching tolerations/nodeSelectors (or affinity rules) on your batch jobs (Airflow KubernetesExecutor pods, training jobs) versus your API/agent deployment.
- Write this design down in docs/node-isolation.md before implementing it on EKS next week — this is exactly the kind of platform-engineering judgment that reads as senior, and it's worth having a clean written rationale for.

Deliverable: docs/node-isolation.md with a written, defensible node-group isolation design, ready to implement on EKS.

Week 13 — EKS: the real cloud deployment

Day 60 Provision EKS with two node groups

- Provision EKS via eksctl (or Terraform if you have the time budget and want the IaC story — same guidance as the finance project: don't let learning Terraform from scratch eat this phase).
- Provision two node groups per your Day 59 design — labeled/tainted distinctly (e.g., workload-type=batch vs workload-type=inference) — rather than one undifferentiated pool.
- Confirm kubectl reaches the cluster and both node groups show up correctly labeled.

Deliverable: A running EKS cluster with two distinctly labeled/tainted node groups, billing alarm double-checked.

Day 61 Deploy the API service to the inference node group

- Push the API image to ECR, apply manifests with the appropriate nodeSelector/toleration to land it on the inference node group specifically, confirm it's reachable and actually scheduled where intended (kubectl get pods -o wide to check node placement).

Deliverable: The FastAPI/agent service running on EKS, confirmed scheduled onto the inference-designated node group.

Day 62 Deploy Airflow + n8n, batch jobs to the batch node group

- Deploy n8n and the Airflow Helm release to EKS, pointed at RDS.
- Configure Airflow's KubernetesExecutor pod template (or per-task overrides) to schedule onto the batch node group via the appropriate toleration — confirm scheduled pods actually land there, not on the inference group.

Deliverable: n8n and Airflow running on EKS, with Airflow's task pods confirmed landing on the batch node group, not the inference one.

Day 63 Secrets management

- Move credentials (LLM API key, DB password, any third-party keys) into AWS Secrets Manager, synced via External Secrets Operator — same standard, explainable pattern as the finance project, same reasoning (plain base64 Kubernetes Secrets are not a real security boundary).

Deliverable: Sensitive credentials sourced from AWS Secrets Manager, not stored as plain base64 Secrets.

Day 64 Buffer / full EKS smoke test with node-isolation verification

- Run a full pipeline cycle on EKS: Airflow ingests and trains, agent answers a question, n8n triage fires — all on the cloud deployment.
- Explicitly verify the node-isolation story held under real load: confirm batch jobs never landed on the inference node group and vice versa, across this full test run.
- Catch up on slippage; tear the cluster down overnight if pausing.

Deliverable: A full pipeline-to-agent-to-triage cycle verified on EKS, with node-group isolation confirmed correct under a real run.

Week 14 — CI/CD, monitoring

Day 65 GitHub Actions: build and push on merge

- Extend CI: on merge to main, after tests pass, build and push images to ECR with a commit-SHA tag, authenticating via GitHub OIDC federation (not static AWS keys).

Deliverable: CI builds and pushes tagged images to ECR automatically on merge.

Day 66 GitHub Actions: deploy to EKS

- Add a deploy job applying updated manifests/Helm upgrades to EKS with the new image tag, respecting the node-group placement rules already configured.
- Test the full loop: push a change, watch CI build/push/deploy, confirm the new version is running on the correct node group.

Deliverable: A full CI/CD loop verified working end to end, with deployments still landing on the correct node groups.

Day 67 Monitoring and logging

- Wire CloudWatch Container Insights / structured logs for EKS workloads, plus Airflow failure alerting.
- If you have time, add basic Prometheus/Grafana dashboards split by node group (batch vs inference) — this directly visualizes the isolation story you built, which is a nice demo moment.

Deliverable: Logs and failure alerting working, demonstrated via a deliberate test failure.

Day 68 Rollout/rollback story

- Deliberately deploy a broken API version, confirm readiness probes prevent it from serving traffic, then practice kubectl rollout undo (or helm rollback) and confirm recovery.

Deliverable: A demonstrated rollback recovering from a deliberately broken deployment.

Day 69 Buffer / Phase 5 close-out

- Catch up on slippage. If significantly behind overall, this phase (specifically Terraform/IaC depth) is the most negotiable place to cut, same guidance as the finance project — the data, ML, and agent layers are the real differentiators.
- Write docs/deployment.md covering the full CI/CD, Kubernetes, and node-isolation architecture.

Deliverable: Phase 5 fully closed out: CI/CD deploying to EKS automatically, node-group isolation working and documented, monitoring and rollback demonstrated.

Phase 6 — Portfolio Packaging

Goal: turn the build into something a hiring manager can absorb in 5 minutes and respect in 30. Position ResolveIQ as a real-data complaint-operations intelligence platform — the fact that it runs on actual public CFPB data, not synthetic data, is itself a selling point worth stating explicitly, since it proves you can handle the unpredictability of real-world data rather than only ever working with clean, invented inputs.

Duration: 2 weeks (10 days).

Week 15 — Demo, screenshots, metrics

Day 70 Script the demo

- Write a 5-8 minute demo script: open with the real operational problem (triaging millions of real complaints), show the routing/urgency/anomaly outputs on real data, then the agent answering the three example analyst questions live with visible, verifiable citations back to real complaint IDs, then a triage approval flowing through n8n.
- Lead with 'this runs on the actual public CFPB database, updated daily' — that's a genuinely strong opening line, say it early.

Deliverable: *A written demo script.*

Day 71 Record the demo

- Record a screen capture following the script, using real data and a real agent response (not mocked). If you want to make the 'real data' point land even harder, briefly show a live complaint that's recent (e.g., from the last few days, pulled by your daily DAG) alongside the response — it visibly proves the pipeline is live, not a one-time snapshot.

Deliverable: *A recorded demo video, 5-8 minutes.*

Day 72 Architecture deep-dive writeup

- Write the architecture deep-dive: bronze/silver/gold flow over real CFPB data, the three models and their real metrics (macro-F1, PR-AUC, precision@k), the agent's citation-enforcement and unverified-narrative-hedging design, the n8n Wait-node triage pattern, and the EKS node-isolation story.
- Assemble this from the docs/*.md files you've kept up through the build — same discipline as the finance project, assembly not first-draft writing.

Deliverable: *A complete architecture deep-dive document.*

Day 73 Metrics one-pager + screenshots

- Pull every concrete real metric into a one-pager: routing macro-F1, urgency PR-AUC, anomaly precision@k, agent groundedness score, plus real scale numbers (actual rows ingested, actual date range covered, actual narrative corpus size).
- Clean screenshots of the routing/urgency/anomaly outputs and the agent's cited answers.

Deliverable: *A metrics one-pager with real numbers and a folder of clean screenshots.*

Day 74 Buffer / non-negotiables self-check

- Re-confirm the same three non-negotiables from the finance project apply here and are genuinely true: evaluation defined from the start (yes — macro-F1/PR-AUC/precision@k chosen deliberately per problem), full audit trail (agent_audit_log + n8n decision logging), human-gated actions (every triage routes through approval before being treated as actioned).

- Add a fourth, project-specific non-negotiable worth checking: does the agent correctly hedge on unverified narrative claims in every test case, not just the one you built on Day 43? Re-run a few spot checks.

Deliverable: A written self-check confirming all non-negotiables, including the narrative-hedging behavior, with evidence for each.

Week 16 — Repo polish, resume, final review

Day 75 README as the front door

- Rewrite the README: a one-paragraph pitch leading with the real-data point, an architecture diagram, a quickstart (docker compose up -d, with a note that the API key is free for CFPB itself but the LLM provider key is needed for the agent), links to the demo and deep-dive.
- Actually test the quickstart on a clean checkout.

Deliverable: A polished README a stranger could follow to run the project locally.

Day 76 Resume bullets and portfolio update

- Draft resume bullets covering: the real-data ingestion/Airflow/RDS pipeline; the three ML models with real metrics and honest imbalance handling; the citation-enforced agent with n8n human-in-the-loop triage and EKS node-isolation deployment.
- If you've also built the Revenue-to-Close finance platform, frame the two together in your portfolio narrative — one shows internal financial-operations automation, this one shows external consumer-complaint operations with real public data; that pairing is a genuinely strong, coherent story for fintech/RegTech/complaint-ops-adjacent roles specifically.

Deliverable: Finalized resume bullets and updated portfolio presence.

Day 77 Mock interview prep

- Write your own answers to likely hard questions: 'How do you know your anomaly detector isn't just flagging the known 2024-2025 surge?', 'Why precision@k instead of recall for anomaly flags?', 'How do you stop the agent from treating an unverified complaint narrative as fact?', 'Why separate node groups for batch vs inference — what would break without that?', 'What would change if CFPB's API rate-limited you tomorrow?'
- These should be specific, one-paragraph, grounded in decisions you actually made.

Deliverable: Written answers to 8-10 anticipated hard interview questions.

Day 78 Tag a release, final cleanup

- Clean up dead code and stale TODOs, run a secrets scan (gitleaks) across history, tag a v1.0 release with release notes.

Deliverable: A clean, secret-free repo tagged v1.0 with release notes.

Day 79 Final buffer — done is done

- Use this day for anything still open. At the end of it, the project is done — same closing principle as the finance project: a finished, honestly-scoped platform beats a perpetually-almost-finished bigger one.

Deliverable: ResolveIQ: complete, deployed, documented, and demo-ready, running on real CFPB data updated daily.

Reference: Risk Register

Known friction points worth re-reading before you hit them.

Risk	Where it bites	Mitigation
EKS/RDS cost creep	Phases 0, 5	Billing alarm from Day 1; tear down EKS clusters when not actively working; small instance sizes
Mistaking the volume surge for anomalies	Phase 3, Day 30	Explicitly detrend/normalize for the documented 2024-2025 surge before flagging anything as anomalous
Treating narratives as verified fact	Phases 1, 3, 4	Bake 'consumers reported X' framing into agent prompts and your own writeup; narratives are opt-in, unverified, PII-scrubbed self-reports
Categorical taxonomy drift over time	Phase 1, Day 7	Decide explicitly whether to map historical categories to current ones or accept regime breaks in your time series; document the choice
LangChain ToolStrategy + extended thinking conflict	Phase 4, Day 40-41	Known, documented friction on Anthropic models specifically; disable extended thinking for structured-output calls or use ProviderStrategy where supported
n8n Wait-node bugs in older versions	Phase 4, Day 45	Use n8n v2.0+ — the sub-workflow Wait-node fix specifically affects HITL gates inside agent-run sub-workflows
Company-name and category noise	Phase 1, 3	Accept and document the noise rather than over-engineering fuzzy matching for a portfolio project, unless time allows
UI polish eating time that should go to depth	Phases 4-6	Cut UI before deployment, testing, auditability, or evaluation if time is short — same principle as the finance project

Reference: Skills Map (same baseline, real-data context)

Skill	Where it appears in ResolveIQ
SQL	Complaint marts, daily volume aggregation, anomaly queries, agent SQL tools
Python	CFPB API client, Airflow DAGs, transformations, model training, FastAPI services, LangChain tools
Pipelines	Bronze/silver/gold flows, daily incremental + historical backfill DAGs
ETL / ELT	Raw CFPB pulls, categorical normalization, SQL-first transformations
Airflow	Scheduled orchestration, backfills, data-quality tests, model-training schedules
AWS	S3, RDS PostgreSQL, EKS (with isolated node groups), ECR, CloudWatch
Docker	Local development, reproducible services, containerized deployment
LLMs	Narrative summarization, theme extraction, grounded analyst Q&A
LangChain	create_agent, content_and_artifact retrieval, enforced citation schemas via ToolStrategy
Vector databases	pgvector over real complaint narratives with metadata-filtered HNSW retrieval
CI/CD	GitHub Actions running tests, building images, deploying to EKS via OIDC
Kubernetes	Production deployment, isolated batch vs. inference node groups, rollout control
n8n	Wait-node HITL triage, approval/reassign/dismiss gates, timeout escalation
Agentic infrastructure	Scoped tool functions, audit logs, citation enforcement, defensive prompting against injected narrative text
ML model training	Multiclass routing classifier, urgency/timeliness classifier, time-series + narrative anomaly detection

Reference: Phase-by-Phase Timeline Summary

Phase	Weeks	Days	Core output
0 — Environment & first API call	Week 1	1-4	Repo, Docker stack, AWS account, CI skeleton, working CFPB pull
1 — Schema & real data exploration	Week 2	5-9	Canonical ERD informed by real exploration, categorical mapping decisions
2 — Data platform	Weeks 3-4	10-19	Incremental + backfill Airflow DAGs, canonical schema in RDS, gold marts
3 — Intelligence layer	Weeks 5-7	20-34	Routing classifier, urgency classifier, anomaly detection — all evaluated on real imbalance
4 — Agent layer	Weeks 8-11	35-54	pgvector RAG, citation-enforced agent, n8n Wait-node triage, hardened API
5 — Production deployment	Weeks 12-14	55-69	Kubernetes, EKS with isolated node groups, CI/CD, secrets, monitoring
6 — Portfolio packaging	Weeks 15-16	70-79	Demo video, architecture writeup, polished README, resume bullets

Total: 16 weeks of active building (~79 working days at 4 hrs/day) plus the buffer days already built into each phase — roughly one week shorter than the Revenue-to-Close finance project, almost entirely because there's no synthetic-data-generation phase to budget for. Realistically, expect this to span 4-4.5 calendar months with normal life interruptions.

The same closing point applies here as it did for the finance project, with one addition specific to working on real data: the schedule is a tool, not a contract, and real data will occasionally force a detour the plan didn't anticipate — a taxonomy quirk, an unexpected null pattern, a surge you have to explain. Treat those detours as the actual value of having chosen real data over synthetic data, not as setbacks. The parts worth protecting if time gets tight are the same as before — audit trail, evaluation rigor, human gating, and now also the honest treatment of unverified narrative content as exactly that. Everything else (UI polish, IaC depth, full historical backfill) is negotiable; those four are not.